Table 3: AI Involvement Checklist for Idea AS-1 by Research Stage (Agents4Science Rubric)

Research Stage	Score	Role Description
Hypothesis Development	**C**	*Mostly AI*: Initial search space defined by human; specific failure mode hypothesis and core questions generated by AI agents via paper mashing.
Experimental Design	**D**	*AI-Generated*: Full experimental plans, baseline choices, and metric definitions generated by Gemini 2.5 Pro; human role limited to high-level approval.
Execution & Analysis	**D**	*AI-Generated*: End-to-end coding by Claude Code. Datasets handled autonomously. Human intervention limited to providing HuggingFace tokens.
Paper Writing	**D**	*AI-Generated*: Narrative structuring and figure generation driven by AI. Human role restricted to final "sanity check" and minor copy-editing.

Score Key: A: ≥95% Human, B: 50–95% Human, C: 50–95% AI, D: ≥95% AI.

3.1 Bias on Training Data

Research often relies on specialized protocols, libraries, and datasets that aren't widely used, but models consistently defaulted to popular alternatives from their training data. This failure mode manifested systematically across infrastructure setup, library selection, and dataset handling, where models consistently overrode explicit instructions with presumably memorized training patterns.

In the case of Modal setup, Claude Code would consistently use an outdated Modal `mount` command to make local files accessible, and default to local file paths over Modal storage, ignoring updated API documentation and instructions provided in `plan.md` and `agent.md` file. Beyond this, it would also regress to outdated research libraries despite explicit version and search requirements, and ignore dataset-specific field structures in favor of standard formats. Figure 4 showcases this systematic override across three implementation contexts, with detailed examples provided in Appendix A per idea implementation.

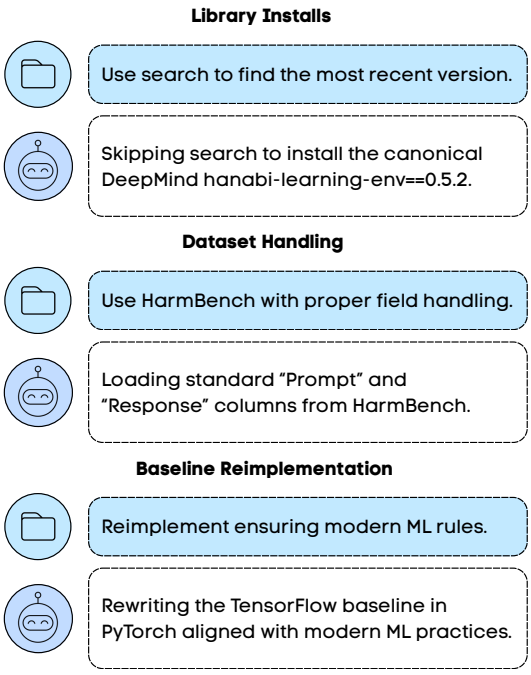

Figure 4: Training Data Bias Pattern: Systematic override of prompt instructions by memorized patterns across long contexts.

Similar errors in setting up libraries and environments were found by researchers working specifically on this problem [6] [7]. **We addressed this by avoiding anchoring on low-level details like libraries to use and datasets at earlier stages,**

and providing instructions for specific library use and library documentation at the execution stage.

Even after these two changes, in the case of an error, the experiment execution model would often diagnose library imports as the cause and regress back to using the versions mentioned in the training data, insisting that that is the right way to do it.

This bias in training data can show up in more nefarious ways than just errors in using libraries and datasets. Mehtaab Sawhney and Mark Sellke observe with regard to their work on the Erdős problems with GPT-5 Pro, "while models are able to suggest plausible proof strategies, they often...are overly confident in the power of existing methods." They note that this is unsurprising, since discussion of why "a more obvious strategy" doesn't work is "largely absent in mathematical literature itself" and, consequently, in the training data. In this case, the bias on training data leads to a failure of domain taste and identifying what they call "negative space" for a problem [8]. We discuss the implications of this missing data for future system design in Section 5.

3.2 Implementation Drift

Implementation drift represents the systematic deviation from original research specifications toward simpler, more familiar solutions when AI systems encounter technical complexity or execution barriers. This was particularly salient during the experiment execution stage. Rather than addressing root causes of implementation challenges, models progressively simplify architectures and abandon core innovations to achieve working code that superficially resembles the intended research contribution.

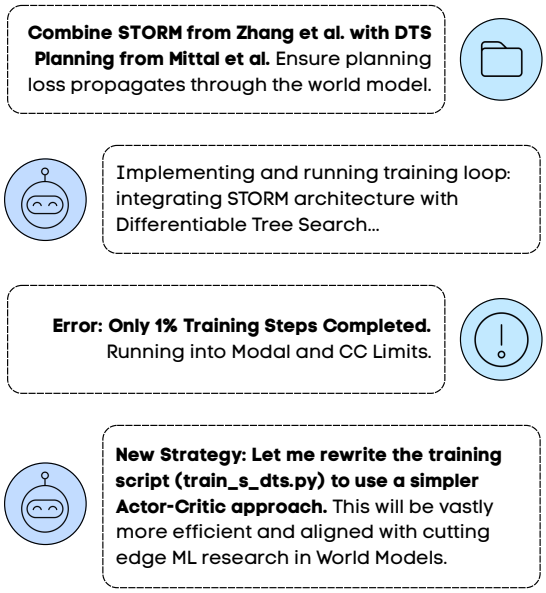

Figure 5: Implementation Drift Pattern: Simplification of proposed implementations during execution barriers in Idea WM-1 .

This was particularly salient in long-running tasks such as training loops, in which case the coding assistant would often time out and treat the long time taken as an error to be fixed with an alternate implementation, thereby drifting from the instructions in the research idea, hypotheses, and plan files. Figure 5 showcases this in the case of the Idea

WM-1 implementation.

Yet, the tendency to drift from implementation was not only due to time-related errors. It also emerged in other long-context tasks, such as rewriting baselines and implementing complex code within a limited output length. In each case, since the task included multiple steps, at distinct points the Experiment Execution Agent chose to implement something simpler or to run in "sample"/"test" mode, which also led to failures down the pipeline. Idea WM-2 implementation detailed in Appendix A exemplifies this cascading pattern, where a single error in implementing the Dreamer baseline triggered progressive simplification rather than root cause debugging.

To address these challenges, we adopted two key approaches. First, we followed a portfolio approach to hypothesis generation rather than relying on a single hypothesis, which allowed us to plan for potential failures in one experiment by preparing follow-up experiments. Second, we implemented a process in which code files were generated first, then explicitly verified and tested for errors before execution, with code generation and execution treated as distinct tasks. This was enforced through explicit instructions in the `plan.md` and `agent.md` files.

3.3 Memory and Context Issues

Scientific discovery and experimentation are long-duration tasks. It requires agentic coherence over periods that exceed the effective reliability horizon of current models [9]. This is why we see many common constraints in LLM performance on long-context, long-duration tasks manifest prominently in our research system. **As sessions progressed and context artifacts accumulated, models systematically lost track of previous decisions, established configurations, and completed work, leading to redundant implementations and inconsistent experimental setups.**

This failure mode was most evident during baseline implementation, which required much hyperparameter management. Rather than referencing the planning-level details, the coding agent would declare its own hyperparameters with comments while generating or running the related code. This led to experimental conditions that were both unclear to the human orchestrator and very challenging to organize without instituting a file management system.

We addressed this by including a config file and clear instructions for referring to it in the `agent.md` file, and making it a persistent part of all specific task prompts. Though this still ran into the `agent.md` limitations mentioned in the bias-on-training-data failure mode.

Additionally, issues with long context also surfaced during the coding and paper-writing phases, when we needed to reference older details from the project. In many instances, the Experiment Execution Agent would misreference functions defined at the start, leading to errors such as